Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Supporting Self-Descriptiveness, Reusability, and Code Bidirectionality in Spreadsheet Programming

Bachelor Thesis

Robin Frey

January 06, 2026

Advisors: Prof. Dr. Z. Su, Dr. S. Thorgeirsson
Department of Computer Science, ETH Zürich

Acknowledgments

You can use this section to thank whoever you want to in the context of your thesis. This can of course include family and friends, colleagues, your advisor, and in general anyone that provide some form of support. You can also take the opportunity here to give credit to people who may have enlightened you with discussions, and also certainly your co-authors of an academic paper related to the thesis, should there be one.

Abstract

Contents

Contents iii

1 Introduction 1

2 System Design 2
- 2.1 Architectural overview 2
- 2.2 The presentation layer 3
  - 2.2.1 Flow graph justification 3
  - 2.2.2 Layout strategy 3
  - 2.2.3 The Datatable view 3
  - 2.2.4 Node design 3
  - 2.2.5 Interaction design 3
- 2.3 The visualization layer 4
- 2.4 The data layer 4
  - 2.4.1 Formula engine selection 4
  - 2.4.2 Single engine instance 5
  - 2.4.3 Asymmetric data flow 5
  - 2.4.4 Data Structure 5

3 Notation and Preliminaries 6

4 Conclusion 9

Bibliography 11

Dummy Appendix 12

Chapter 1

Introduction

Motivation

Spreadsheets are prone to weak points known as Spreadsheet smells. Common smells are Feature Envy, where a formula accumulates excessive dependencies, and Inappropriate Intimacy, where data is tightly coupled across different worksheets [3]. (TODO: build reusability into it) These smells hinder comprehension and make it harder for the users to find information. Readers of Spreadsheet spent 40% of their time seeking for information, such as locating items, understanding interactions, tracing formula logic and understanding unfamiliar spreadsheet concepts [4]. Improving Self-descriptiveness is therefore critical to reduce the time spent seeking for information. However visualization alone is insufficient. The lack of Bidirectionality forces users to constantly switch context on these information detours, further increasing the time and cognitive effort required.

Previous Work

Our Contribution

Thesis Outline

Chapter 2

System Design

2.1 Architectural overview

To ensure strict separation between visualization and calculation the system implements a layered component architecture.

1. The presentation Layer
2. The state and Context Layer
3. The business Logic Layer
4. The data Layer

The **presentation** Layer is responsible for interactions and rendering. It implements a split-view displaying the datatable and flow-graph simultaneously. This layer only presents the data from the state layer and dispatch ssers interactions. It does not directly manipulate the data. In the second layer, the **state & context Layer**, ...

The **business logic layer** encapsulates all the transformer to enable the bidirectionality. For the Spreadsheet to flow graph direction, first the parser translates an unstructured formula String into an AST[1]. To bridge the gap between AST and flow graph we need another transformer which maps elements of the AST to elements of the flow graph. The use of an intermediate AST was deliberately chosen over raw string manipulation to ensure correctness when handling complex and deeply nested formulas for a very small performance cost. For the inverse direction, going from flow graph to a formula string, is the serializer responsible. The foundation of the architecture, the **data layer**, manages all operations of the datatable. By delegating every operation to this layer, the system ensures consistency and therefore provides synchronization between the two views.

[1] Abstract Syntax Tree

2.2 The presentation layer

For the presentation layer there are 3 important goals:

1. improve self-descriptiveness of the hidden data of the datatable
2. improve bidirectionality between the tool and the datatable
3. minimize the cognitive load of the user

2.2.1 Flow graph justification

The decision to represent the hidden data as a flow graph was driven by the nature of the underlying state of spreadsheet formulas, the dependency graph. A flow graph allows us to uncover this state and enables the user to see the dependencies rather than by reading formulas and parsing it on their own. The flow direction is left to right. This should mirror the spatial convention for spreadsheets, where the inputs of a formula are located in columns on the left side of the formula.

2.2.2 Layout strategy

Placing the datatable-view and flow-graph-view side by side serves two purposes. First by having them as near as possible to each other it should reduce the cognitive distance between datatable-view and flow-graph-view. Secondly by strictly separating the both views i.e not having a popover or even an overlay strategy, prevents the occlusion of other important cells.

2.2.3 The Datatable view

2.2.4 Node design

2.2.5 Interaction design

The flow graph not only serves as a visualization but also as an interface for the exploration and manipulation of the datatable. Interactions are organized into two distinct modes: the **preview** mode and the **edit** mode. This separation should provide clear visual feedback when changes will affect the underlying data of the datatable.

Preview Mode

The Preview Mode is the default state and is intended for the exploration. Hovering over a reference node highlights the corresponding cell or range in the datatable. This allows the user to survey the dependency at the current position in the datatable. Clicking on a reference node scrolls the datatable to the corresponding cell or range to enable closer inspection of the dependencies.

Edit Mode

Edit Mode is always entered through an explicit user action:

1. clicking on the edit button
2. double-clicking on an inline value or address
3. double-clicking an edge
4. dragging from a connection handle

The entry action determines the initial editing context. Double-clicking on a node's address allows the user to select a new address directly from the datatable. The change is committed by pressing enter or clicking the confirmation symbol. Double-clicking an inline value transforms the field into an editable input. The user modifies the constant and commits the change through the same confirmation mechanism. Double-clicking an edge enters the edit mode with the double-clicked edge selected for further manipulations. Dragging from a connection handle initiates edge creation or modification. The user drags from a source handle to a target handle. While dragging the edge invalid handles are grayed out. If the target handle has no existing connection, the new edge connects directly. Else the target handle already has a connection, a popover asks the user to either replace the existing edge or swap positions with it. Clicking only the edit button enters the edit mode without an initial editing context. Once in edit mode, the user can change context freely.

2.3 The visualization layer

2.4 The data layer

The data layer forms the foundation of the system architecture. It serves as the single source of truth for all spreadsheet related data. This includes for example: cell values, formulas and computed results.

2.4.1 Formula engine selection

Definition 2.1 *A **worksheet** is a two-dimensional grid of cells that are organized into rows and columns. [2, p. 65]*

Definition 2.2 *A **workbook** is a collection of worksheets. [2, p. 65]*

For the formula engine it is important that it provides a lot of functions which are also used in Microsoft Excel or Google Sheets to make the user as familiar with the formulas as possible. Another important feature should be the ability to handle multiple worksheets. The system uses HyperFormula

as its calculation engine. HyperFormula provides compatible syntax with Microsoft Excel, 400 built-in functions and an easy integration with custom front-ends [1]. This allows the user to transfer their pre-knowledge about spreadsheet functions and behavior to this system.

2.4.2 Single engine instance

A single HyperFormula instance maintains the whole calculation state for the entire workbook. This allows the user to reference cells and ranges from different worksheets.

Definition 2.3 ***Dependants*** *of a cell address or range are :*

1. *All cells with formulas that contain the given cell address or range.*
2. *Some of the ranges that contain the given cell address or range.*

[1]

Having only one single-instance implies having only one dependency graph for all worksheets. This simplifies the process of checking and updating dependencies. But this also implies that one computationally heavy formula could affect the responsiveness of the entire workbook.

2.4.3 Asymmetric data flow

As mentioned in the presentation layer 2.2 each worksheet is implemented by their own Handsontable component. Each component reads/writes from/to the same Hyperformula Engine. All components of the Graph-View also read directly from the same Hyperformula Engine. A read is done for example when nodes calculate their intermediate results. In contrast to the read paths, data manipulation in the Graph-View follows a slightly different path. Because HyperFormula is a headless calculation engine it is not able to know if it is connected to any Handsontable component. Therefore to prevent inconsistencies between the datatable and the underlying calculation engine, edits to the data have to go through the active Handsontable component to trigger rendering and the corresponding hooks.

2.4.4 Data Structure

Chapter 3

Notation and Preliminaries

Definitions and Notation

Arrays and Median

Throughout this thesis, we denote the input array of size n by

$$a = (a[1], a[2], \ldots, a[n]).$$

We may also use $a[1 \ldots n]$ to denote an array a of length n. Our goal is to find the *median* of a. Formally, we define:

- If n is odd, the median is the element in the $\lceil n/2 \rceil$-th position of the array when sorted in non-decreasing order.
- If n is even, the median is the element in the $\lceil n/2 \rceil$-th position of the sorted array.

Random Variables and Expectation

In the analysis of our randomized median-finding algorithm, we shall frequently refer to *random variables*. A random variable X is a function from the sample space Ω (representing all possible outcomes of a probabilistic experiment) to the real numbers $\mathbb{R}$.

Expectation. The *expected value*, or *mean*, of X is defined by

$$\mathbb{E}[X] = \sum_{\omega \in \Omega} X(\omega)\, \mathbb{P}[\omega],$$

in the discrete case. If X has a probability density function f_X (in the continuous case), then

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x\, f_X(x)\, dx.$$

Intuitively, $\mathbb{E}[X]$ indicates the "average" or "central" value of the random variable over many independent trials.

Variance and Standard Deviation

To measure how much a random variable deviates from its mean, we use its *variance*, denoted $\mathrm{Var}[X]$. This is defined as:

$$\mathrm{Var}[X] = \mathbb{E}[(X - \mathbb{E}[X])^2].$$

The *standard deviation* of X, denoted σ_X, is the square root of the variance:

$$\sigma_X = \sqrt{\mathrm{Var}[X]}.$$

A smaller variance (or standard deviation) means the random variable's values tend to lie closer to its mean, whereas a larger variance indicates a broader spread of values.

Asymptotic Notation

In describing the time complexity of algorithms, we use $\mathcal{O}(\ldots)$ notation in a standard way:

$$f(n) = \mathcal{O}(g(n)) \quad \Longleftrightarrow \quad \lim_{n\to\infty} \frac{f(n)}{g(n)} < \infty.$$

We may also use $\Theta(\ldots)$, $\Omega(\ldots)$, and other asymptotic notations where necessary.

Chebyshev's Inequality

Chebyshev's inequality is a cornerstone result in probability theory, providing a quantitative bound on how a random variable deviates from its expected value. Formally, let X be any random variable with finite mean $\mathbb{E}[X]$ and finite variance $\mathrm{Var}[X]$. Then, for any $k > 0$,

$$\mathbb{P}[|X - \mathbb{E}[X]| \geq k] \;\leq\; \frac{\mathrm{Var}[X]}{k^2}.$$

Intuition and Relation to Markov's Inequality

To build intuition, recall the simpler *Markov's inequality*, which states that for any nonnegative random variable Y and $t > 0$,

$$\mathbb{P}[Y \geq t] \leq \frac{\mathbb{E}[Y]}{t}.$$

Chebyshev's inequality is often viewed as an application of Markov's inequality to the random variable $(X - \mathbb{E}[X])^2$. Indeed,

$$\mathbb{P}[(X - \mathbb{E}[X])^2 \geq k^2] \leq \frac{\mathbb{E}[\left(X - \mathbb{E}[X]\right)^2]}{k^2} = \frac{\mathrm{Var}[X]}{k^2},$$

and from $\{\, |X - \mathbb{E}[X]| \geq k \,\} \equiv \{\, (X - \mathbb{E}[X])^2 \geq k^2 \,\}$, we obtain Chebyshev's inequality.

The key takeaway is that Chebyshev's inequality captures a broad class of distributions. It requires no assumption of normality or other specific distributional shapes, only finite variance. In exchange for this generality, Chebyshev's bound can be somewhat loose compared to more specialized inequalities (e.g., Hoeffding's, Chernoff's, or Bernstein's). Nonetheless, it remains a powerful and classic tool for bounding "tail events," i.e. events in which X significantly deviates from its mean.

Why We Use Chebyshev's Inequality Here

In our randomized median-finding algorithm, we draw random samples from the input array and use them to approximate the true median. The "rank" of our chosen pivots in the sorted order of the entire array are themselves random variables. We want to show that, with high probability, these rank containt the $\lceil n/2 \rceil$-th position.

By applying Chebyshev's inequality, we obtain a bound on $\mathbb{P}[|X - \mathbb{E}[X]| \geq k]$ for a well-chosen k, where X is the rank of our pivot. In other words, Chebyshev's inequality formally establishes the reliability of our sampling strategy by demonstrating that, except with small probability, our pivots partition the array around the actual median. This allows us to restrict further computation to a subset of the array whose size is guaranteed to be sufficiently small, thereby ensuring linear running time.

Extensions and Alternatives

While Chebyshev's inequality is sufficient for our purposes, more refined analyses might employ other concentration bounds such as Hoeffding's or Chernoff's inequalities. These stronger bounds can yield tighter probabilities when the underlying sample distribution has additional regularity properties (e.g., bounded random variables or independent Bernoulli trials). However, Chebyshev's inequality has the advantage of simplicity and broad applicability, making it an ideal choice for illustrating the core ideas in a clear manner.

Chapter 4

Conclusion

In this thesis, we presented a randomized selection algorithm for identifying the median of an unsorted array in expected linear time. Our method hinges on sampling a sublinear number of elements, sorting them to obtain pivots, and then "zooming in" on a small region of the array that is guaranteed, with high probability, to contain the true median. The crux of our analysis relied on Chebyshev's inequality, which provided a principled way to bound the probability that our random pivots deviate significantly from the actual median's rank in the sorted order.

Summary of Contributions.

- We introduced a randomized median-finding algorithm that leverages a sample of size roughly $n^{3/4}$. Through sorting this sample and selecting pivots near its middle, the algorithm narrows down the search space to a sub-array whose size is no more than $4n^{3/4}$.
- We proved a high-probability bound (Theorem **??**) on the correctness of the algorithm. Specifically, with probability at least $1 - \mathcal{O}\left(n^{-1/4}\right)$, our pivots capture the true median, ensuring that the algorithm does not report *failure*.
- We established a near-linear worst-case running time (Theorem **??**). The cost of sorting the small sample and, conditionally, the sub-array T both remain asymptotically lower than $\mathcal{O}(n)$, leaving the dominant cost to be a single pass over the array to compute element ranks relative to the pivots. Hence, the overall algorithm runs in $\mathcal{O}(n)$ time with high probability.
- Beyond the immediate theoretical guarantees, we discussed how to combine our randomized approach with a deterministic "median-of-medians" fallback to handle rare failure events, ensuring correctness

remains absolute. This hybrid strategy preserves the practical advantages of randomization while delivering a worst-case linear-time bound.

Limitations and Possible Extensions. While our use of Chebyshev's inequality is conceptually simple and broadly applicable, tighter concentration results (e.g. Chernoff or Hoeffding bounds) could potentially yield smaller failure probabilities or allow an even smaller sample size. However, such refinements might require more specific assumptions about the distribution of the input data or the independence structure of the sampling process.

Another direction for future exploration is to extend the approach to streaming and dynamic settings. In scenarios where the dataset arrives incrementally, maintaining a small, representative sample to approximate the median could be achieved through reservoir sampling or related techniques. Additionally, one might investigate parallel or distributed variants of the algorithm, where the sampling and pivot-selection steps are split across multiple processors or machines.

Bibliography

[1] HyperFormula (v3.1.1).

[2] ECMA-376-1:2016 Office Open XML File Formats - Fundamentals and Markup Language Reference, 2016.

[3] Felienne Hermans, Martin Pinzger, and Arie van Deursen. Detecting and visualizing inter-worksheet smells in spreadsheets. In *2012 34th International Conference on Software Engineering (ICSE)*, pages 441–451, June 2012. ISSN: 1558-1225.

[4] Sruti Srinivasa Ragavan, Advait Sarkar, and Andrew D Gordon. Spreadsheet Comprehension: Guesswork, Giving Up and Going Back to the Author. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, pages 1–21, Yokohama Japan, May 2021. ACM.

Dummy Appendix

You can defer lengthy calculations that would otherwise only interrupt the flow of your thesis to an appendix.

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. **In consultation with the supervisor**, one of the following two options must be selected:

☐ I hereby declare that I authored the work in question independently, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies[1].

☐ I hereby declare that I authored the work in question independently. In doing so I only used the authorised aids, which included suggestions from the supervisor regarding language and content and generative artificial intelligence technologies. The use of the latter and the respective source declarations proceeded in consultation with the supervisor.

Title of paper or thesis:

Authored by:
If the work was compiled in a group, the names of all authors are required.

Last name(s):	**First name(s):**

With my signature I confirm the following:

- I have adhered to the rules set out in the Citation Guidelines.
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

Place, date	**Signature(s)**

If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.

[1] For further information please consult the ETH Zurich websites, e.g. https://ethz.ch/en/the-eth-zurich/education/ai-in-education.html and https://library.ethz.ch/en/researching-and-publishing/scientific-writing-at-eth-zurich.html (subject to change).